Week 9: Stochastic Methods

Emma James

1 Introduction

> **Learning objectives**
>
> As a reminder, the learning objectives for this week are:
>
> 1. Describe the role and utility of stochastic methods in statistical analysis
> 2. Outline the logic and procedure of bootstrapping
> 3. Explain how and why the non-parametric bootstrap avoids distributional assumptions
> 4. Use for loops and sampling in R to bootstrap a range of statistical parameters
> 5. Relate these stochastic approaches to a range of statistical tasks

In this week's practical, we'll see how stochastic techniques can be used to quantify uncertainty in our parameter estimates via bootstrapped 95% confidence intervals. We will do this first using data that you simulate yourself, and then using data from a study on maternal wellbeing. We'll apply the same techniques to estimate the number of participants needed to detect an effect in a future study (i.e., a *simulation-based* power analysis).

Along the way, we will practice using a very useful feature in programming languages: loops! These allow us to repeat tasks multiple times, and increase the efficiency of our work. The very first section will introduce different types of loops before we begin to use them, but feel free to skip this section if you are already familiar.

---

2 Setting up

2.1 Packages

Today's practical will use the following packages. You will need to install each one, before loading them at the top of your script.

```
library(tidyverse)
```

---

3 Introduction to loops

Loops are a way of getting a computer to do something lots of times. There are two main types of loop: ***for* loops** that complete a task a fixed number of times, and ***while* loops** that continue until one or more conditions are met.

3.1 *For* loops

A *for* loop changes the value of a variable each time around the loop, according to the sequence specified. Here is the simplest possible *for* loop - what do you think it does?

```
# Simple loop
for (i in 1:10) {
  print(i)
}
```

Type this code into your script and execute it. You should get a list of the numbers 1 to 10 in the console. In each iteration, the loop is assigning the value of i to the next value from the list— here a list of numbers from 1 to 10. Each time round the loop, the commands inside the {curly brackets} are repeated. In this case, we're just printing the value of i to the console.

We can scale this up to do more complex things. For example, we could generate some random numbers and put their mean into a list. To do this, we first need to create a vector to store the output. Then, through each iteration of the loop, we calculate the mean of 20 randomly generated numbers and store it in the ith entry of the vector.

```
# Create an empty vector to store output
mean_list <- NULL

# In each of 10 runs through the loop
for (i in 1:10){

  # Generate 20 random numbers from a normal distribution
  random_n <- rnorm(20)

  # Calculate the mean
  mean_val <- mean(random_n)

  # Store the mean value in the corresponding (i) vector entry
  mean_list[i] <- mean_val
}

# Print the output
mean_list
```

Run this section of code: you should see it outputs a list of means, and that you now have this vector of means stored in your R environment. This simple process can be very useful for bootstrapping and other stochastic methods.

> **Tip!**
>
> Note that we wrote each step out in full in the above loop to illustrate each step in turn. However, you could write the contents of the loop much more concisely by nesting the data generation function inside the mean function and assigning it directly to the output vector: `mean_list[i] <- mean(rnorm(20))`.

3.2 *While* loops

An alternative then is the *while* loop. Instead of running for a fixed number of times, the loop terminates only when a certain condition is met. Here is a very simple example:

```
i <- 0                # start with i = 0

while (i < 10){       # providing i is less than 10
  i <- i + 1          # increase i by 1
  print(i)            # print new value of i
}
```

This does exactly the same as the first *for* loop example: it repeats ten times, increasing the value of i each time and printing it out. Although it takes more lines of code, it can be used in interesting ways.

Try modifying the code above so that instead of increasing by a fixed value, it adds a random number generated from a normal distribution. (using the `rnorm()` function). How many times do we go round the loop when the mean of the normal distribution is 0, versus when it is 0.5?

---

4 Part A: Core example

Now that you understand the basics of loops, let's use them to bootstrap the mean of some data.

4.1 Simulating a dataset

As an exception today, we are going to simulate some data to work with in Part A. This is a useful thing to be able to do as it gives us full control over the properties of the data. Here, we'll make sure that the data are not normally distributed by generating them from an exponential distribution. It's also fitting to use random number generation functions in a practical about stochastic methods!

Generate your dataset by using the `rexp()` function. Find out how it works by typing `help(rexp)`. Then create a variable, `a`, with 40 samples from this distribution, setting the rate parameter to 1.

Plot a histogram of your dataset (using either `hist()` or `ggplot()`), and calculate the true mean of the simulated data.

```
# Example histogram
hist(a, col = "purple")

# True mean
mean(a)
```

Note that each of you will have simulated your own dataset using random numbers from a distribution, and so you will each have a different histogram and slightly different true mean!

> **!** Reproducible randomness
>
> You might be thinking that randomness is all very well and good, but what if we (or others) need to be able to reproduce the results? We can do this by specifying the *random seed* used to initialise the random number generator. You can set the seed to any integer that you like (I often use the date!), meaning that the same random numbers will be reproduced next time you run the script. Try using `set.seed(240426)` before generating your dataset - you should now have the same set of random numbers as your peers, and a mean of 1.19.

4.2 Run the bootstrapping

Now let's resample the data many times in a loop, and compute the mean each time. Start by creating an empty variable to store the means in. Let's call it `all_means`:

```
all_means <- NULL
```

Then create a *for* loop that will repeat 1000 times. Inside the loop, we need to do two things: (1) resample with replacement from our synthetic data (`a`); and (2) calculate the mean of the resampled data. We'll store this mean in the vector we created, `all_means`. Embed these two lines inside your loop to perform the resampling on each iteration:

```
b <- sample(a, replace = TRUE)
all_means[i] <- mean(b)
```

Create a histogram of the `all_means` vector. Note that it is normally distributed, despite the underlying data coming from a non-normal distribution. We can get R to work out the 95% confidence intervals using the quantile function:

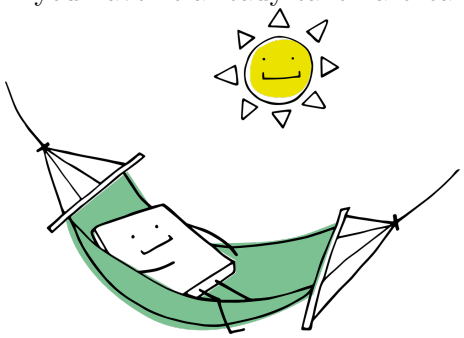
```
ci <- quantile(all_means, c(0.025, 0.975))
```

Print the values of `ci`. These are the bootstrapped 95% confidence intervals of the mean. Try running through the loop and this line several times (but with the same starting sample of data). What do you notice?

```r
## Create empty vector to store bootstrapped means
all_means <- NULL

## Bootstrap means
for (i in 1:1000){
  b <- sample(a, replace = TRUE)
  all_means[i] <- mean(b)
}

## Inspect distribution of bootstrapped means
hist(all_means)

## Calculate 95% confidence intervals
ci <- quantile(all_means, c(0.025, 0.975))
print(ci)
```

Take a break!

If you haven't already taken a break today, we suggest you do so before you start the next section.



---

5 Part B: Apply your knowledge

Let's move on to estimating bootstrapped parameters for inferential statistics, using linear regression as an example. How can we estimate the confidence limits of a regression slope? One way to do this is to resample the data, and perform a separate regression on each resampled dataset. Then we can obtain confidence limits on the slope of the regression line.

5.1 Introduction to the dataset

We will work with data from Dr Catherine Preston's research group. Dr Lydia Munns (a former PhD student in the department) was interested in how bodily experiences during pregnancy are associated

with wellbeing. They administered several questionnaires to >200 pregnant women, including body dissatisfaction and depression which we will look at today. If you're interested in this topic, you can access the full paper here.

We can download the data directly from the OSF. Note that the data are split across two files, so we can join them together as follows. We also need to create a total body dissatisfaction score from the three subscales collected, and select/reformat the names of relevant variables for ease of use today.

```
# Load and combine data files
bumps_dat <- read_csv("https://osf.io/fzn7p/?action=download") %>%
  bind_rows(read_csv("https://osf.io/rux6t/?action=download")) %>%

  # Create total body dissatisfaction score from the three BUMPS subscales
  mutate(body_diss = `BUMPs Physical` + `BUMPs Weight` + `BUMPs Appearance`) %>%

  # Reduce dataset to relevant variables
  select(body_diss, Anxiety, Depression)

# Reformat column names for consistency
colnames(bumps_dat) <- tolower(colnames(bumps_dat))
```

You should now be left with the following variables:

- **body_diss**: body dissatisfaction, as measured by the *Body Understanding Measure for Pregnancy Scale (BUMPs)*
- **anxiety**: anxiety, as measured by the *Hospital Anxiety and Depression Scale*
- **depression:** depression, as measured by the *Hospital Anxiety and Depression Scale*

We will just use the depression scores today, but anxiety scores are retained in the dataset in case you later want to apply what you learn to a new example.

Use R code to answer the following questions about the dataset:

- How many participants completed the surveys?
- What was the mean, median, and range of depression scores?
- How do the variables correlate with each other?

```
# Number of participants
nrow(bumps_dat)

# Depression descriptives - quick way to gain summary scores across all vars
summary(bumps_dat)

# Correlations
cor(bumps_dat, use = "pairwise.complete.obs") %>%
  round(2)
```

5.2 Run a linear regression model

The researchers were interested in how body dissatisfaction during pregnancy predicted depression scores. Fit a simple linear model as follows, then call `summary()` on the fitted object to inspect the output.

```
# Fit a linear model
bumps_lm <- lm(depression ~ body_diss, data = bumps_dat)
```

The intercept and slope of the regression model are stored in the output variable, which you can access in a more direct way using `bumps_lm$coefficients`. You can select the slope specifically using `bumps_lm$coefficients[2]` for the second entry. Run this now and store the entry in your environment as `true_slope`, it should have the value ~0.09. This means that every unit increase in the body dissatisfaction scale is associated with a ~0.1 increase on the depression scale.

We can also use these extracted coefficients to plot a regression line on a scatterplot. Remember that we had some practice in specifying regression lines manually in Week 3 (Quantile Regression).

```
ggplot(bumps_lm, aes(x = body_diss, y = depression)) +
  geom_point() +
  geom_abline(intercept = bumps_lm$coefficients[1],
              slope = bumps_lm$coefficients[2]) +
  theme_classic()
```

5.3 Compute bootstrapped 95% CIs for the slope

To do the bootstrapping, we need to sample the *rows* from the dataframe, so that the relationship between variables is preserved (i.e., the correct pairings between each participant's scores). To do this, we create an index variable that contains the same number of entries as there are rows of the dataframe:

```
# Create data index for resampling
index <- 1:nrow(bumps_dat)
```

Now create a loop to perform 1000 bootstrap samples. Within each iteration, you will need to create a `resampled_index` by resampling the index variable (just as we did when resampling values earlier).

You can then use the resampled variable you have created to extract the corresponding rows from the dataset as follows:

```
# Use resampled index to generate data based on row numbers
resample_dat <- bumps_dat[resample_index,]
```

On each iteration, repeat the regression with the sampled data and store the slope in a vector (as we did for bootstrapping means).

```r
# Create data index for resampling
index <- 1:nrow(bumps_dat)

# Create vector for storing the slope
all_slopes <- NULL

# For each of 1000 iterations
for (i in 1:1000){

  # Resample the row index
  resample_index <- sample(index, replace = TRUE)

  # Use the resampled index to select corresponding rows of the data frame
  resample_dat <- bumps_dat[resample_index,]

  # Fit a new model
  new_mod <- lm(depression ~ body_diss, data = resample_dat)

  # Store the slope coefficient
  all_slopes[i] <- new_mod$coefficients[2]

}
```

Once you have run the bootstrapping, use the `quantile()` function to calculate 95% confidence intervals on the slope.

```r
# Calculate 95% confidence intervals on the bootstrapped slopes
quantile(all_slopes, c(0.025, 0.975))
```

See if you can add the 95% confidence intervals to your figure. You might want use a different linetype and/or colour to distinguish your confidence intervals from your original slope.

```r
ggplot(bumps_lm, aes(x = body_diss, y = depression)) +
  geom_point() +

  # Original regression line
  geom_abline(intercept = bumps_lm$coefficients[1],
              slope = bumps_lm$coefficients[2]) +

  # 2.5% CI
  geom_abline(intercept = bumps_lm$coefficients[1],
              slope = quantile(all_slopes, 0.025),
              linetype = "dashed",
              colour = "blue") +

  # 97.5% CI
  geom_abline(intercept = bumps_lm$coefficients[1],
```

```
              slope = quantile(all_slopes, 0.975),
              linetype = "dashed",
              colour = "blue") +

  theme_classic()
```

5.4 Run a power simulation

Imagine that you want to replicate this finding in a new sample of participants. However, resources are limited, and so you only want to collect data on the minimum number of participants needed to detect the effect.

We can build on our technique above to simulate the number of participants needed for the results to be statistically significant 90% of the time (i.e., we have a 9 in 10 chance of observing the same effect, assuming it is true). We will create 1000 simulated datasets as we did above, and repeat this process for different numbers of participants in our sampling procedure.

Copy and paste your code from above, and make the following edits to achieve this:

- **Storing the results:** Rather than creating an empty *vector* to store the results, it will be more helpful to create an empty *dataframe* that can store both the number of participants sampled and whether the regression coefficients were statistically significant (i.e., the p-value). You can do this by specifying `sim_results <- data.frame(n_ppts = NULL, p_val = NULL)`.
- **Adding a loop:** Wrap your loop inside another *for* loop, this time to vary the number of participants that you are sampling. I suggest starting with `for (n in seq(10, 100, 10)){`, which can be used to increase the number of participants by 10 each time, from 10 to 100 participants.
- **Assigning row numbers:** Indexing the next row of your dataframe using the `i` of the simulation loop will no longer work, as you are running i simulations *per participant number.* This means that after you have run 1000 simulations for 10 participants, your next 1000 simulations for 20 participants would just overwrite your existing results. To get around this, I suggest initiating a row counter *before* you begin your loops (`row_counter <- 0`). Then *within* your loop, you can increase this by 1 in each iteration to identify the next row of your dataframe for storing your results (`row_counter <- row_counter + 1`).
- **Extracting the p-value:** If you have used the same variable names as above, you can extract and store the p-value in your data frame using the following line of code: `sim_results[row_counter, "p_val"] <- summary(new_mod)$coefficients["body_diss","Pr(>|t|)"]` . Adapt the code to store the participant number (n) in the other column.
- **Calculating statistical power:** Once you have run your simulations and have 1000 p-values for each number of participants, you can use *tidyverse* tools to calculate statistical power. You can use `group_by()` to calculate power per number of participants, and `summarise()` the power by calculating the proportion of p-values that were statistically significant at an alpha level of .05 (`mean(p_val < .05)`).

How many participants should you collect data from to have 90% power to detect the predictive relationship between body dissatisfaction and depression scores?

You might find it useful to plot a "power curve" of the summary statistics, showing the number of participants on the x axis and proportion significant on the y axis.

```
# Note that you might well be able to do this more efficiently! :)

# Create data index for resampling
index <- 1:nrow(bumps_dat)

# Create data frame for storing results
sim_results <- data.frame(n_ppts = NULL,
                          p_val = NULL)

# Set row counter
row_counter <- 0

# For each of 1000 iterations
for (n in seq(10, 100, 10)){
  for (i in 1:1000){

  # Resample the row index
  resample_index <- sample(index, size = n, replace = TRUE)

  # Use the resampled index to select corresponding rows of the data frame
  resample_dat <- bumps_dat[resample_index,]

  # Fit a new model
  new_mod <- lm(depression ~ body_diss, data = resample_dat)

  # Identify next row for storing results
  row_counter <- row_counter + 1

  # Store the results
  sim_results[row_counter, "n_ppts"] <- n
  sim_results[row_counter, "p_val"] <- summary(new_mod)$coefficients["body_diss","Pr(>|t|)"]

  }
}

# Calculate power for each set of participants
sim_power <- sim_results %>%
  group_by(n_ppts) %>%
  summarise(power = mean(p_val < .05))
sim_power
## ~70 participants should provide 90% power

# Plot power curve
ggplot(sim_power, aes(x = n_ppts, y = power)) +
  geom_path() +
```

```
  geom_hline(yintercept = 0.9, linetype = "longdash") +
  theme_classic()
```

---

6 Part C: Challenge yourself!

Conducting the bootstrapping process manually helps to show what's really going on behind the scenes, and builds a foundational skill that can be useful in other scenarios—as we have seen with the power analysis simulation.

Ordinarily however, there are often existing functions that can do the bootstrapping for us. Find the documentation for the *boot* package (or another package of your choosing!) and work out how to replicate the bootstrapping procedures above using bootstrapping functions.

---

7 Summary

7.1 Recap

This week we have seen how stochastic methods can be a valuable tool for a range of statistical tasks: we've used randomly generated data to develop and test our statistical approach, and used random resampling methods to calculate bootstrapped parameter estimates. We saw how these approaches can be particularly helpful in cases where normal distributional assumptions are violated, better capturing the asymmetry of confidence intervals around the measure of central tendency.

Looping and sampling approaches can be useful in a wide range of statistical scenarios. We have seen in today's practical that a similar approach can be used for *simulation-based* power analysis to estimate the number of participants needed for a future study. We will continue to build on these ideas next week as we consider ways to address missing data in analyses—so keep these thoughts in mind for then!

7.2 Further learning

You can find this week's core reading on the VLE as usual. Beyond this, there is a whole course on Sampling in R that builds from different types of sampling to their use in bootstrapping robust estimates, or you might enjoy the Quantitude episode on Bootstrapping. If you want to get more practice with loops, there's a whole chapter on them within the DataCamp Intermediate R course.

! Feedback!

These practical sessions are new this year. Please take a moment to rate the difficulty and length of these practical activities via this survey, and let us know of any issues you encountered or aspects you didn't understand (positive feedback is also welcome!). You will need to be logged in with your university account to respond, but your submission will be anonymous.
For more general questions about the practical, please post them on the VLE Discussion Board.